HW3 Report

R13922213 梁正

1. Implementation

- GPU Thread and Block Partitioning

  The task partitioning strategy divides the rendering work using a 2D grid-block hierarchy:

  - Block Configuration: 16x16 threads per block, organized in 2D (threadIdx.x, threadIdx.y)
  - Grid Configuration: Dynamically calculated to cover the entire image:
    - gridSize.x = (width + 15) / 16
    - gridSize.y = (height + 15) / 16
  - Task Assignment: Each thread handles one pixel. The kernel uses block and thread indices to compute the global pixel coordinates:
    - x = blockIdx.x * 16 + threadIdx.x
    - y = blockIdx.y * 16 + threadIdx.y
  - Workload Distribution: The 2D decomposition naturally maps to the image's spatial layout, ensuring cache-friendly memory access patterns and efficient parallel execution across the GPU's streaming multiprocessors.

- Constant Memory

  A critical optimization uses CUDA constant memory to store rendering parameters that remain unchanged across all threads:

  - Parameters like camera position (ro, ta), field of view (FOV), Mandelbulb iteration count (md_iter), and ray marching parameters are stored in the constant memory struct C
  - The host populates a host-side mirror h_C and copies it to device constant memory via cudaMemcpyToSymbol()
  - This eliminates redundant global memory accesses and provides cache-efficient access for read-only data accessed by all threads

- Additional Optimization

  - Memory Coalescing: Linear array indexing (idx = y * width + x) ensures sequential global memory access patterns for raw_image writes
  - Inline Functions: Instead of jumping to a function and back, the compiler substitutes the function body directly at the call site. With inlining, the compiler can:
    - Perform better instruction scheduling across the expanded code
    - Optimize surrounding code in context with the function body

- pow_x(): Custom integer exponentiation using bit manipulation instead of expensive glm power functions

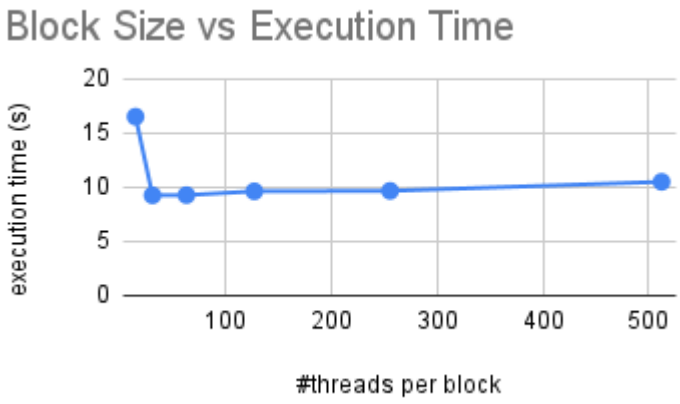
2. Analysis

- GPU kernel execution time using nvprof

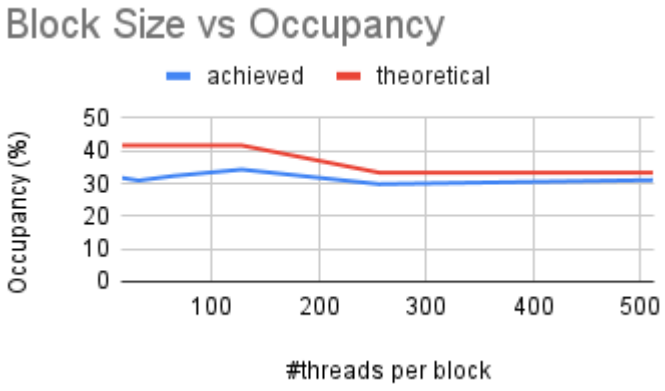
  “render” kernel execution time on TID04 = 2.69 (s)

```
==1109791== NVPROF is profiling process 1109791, command: ./hw3 3.726 0.511 -0.096 0 0 0 2048 2048 output_gpu.png
==1109791== Profiling application: ./hw3 3.726 0.511 -0.096 0 0 0 2048 2048 output_gpu.png
==1109791== Profiling result:
            Type  Time(%)      Time     Calls       Avg       Min       Max  Name
 GPU activities:   99.78%  2.68781s         1  2.68781s  2.68781s  2.68781s  render(int, int, unsigned char*)
                    0.22%  5.8860ms         1  5.8860ms  5.8860ms  5.8860ms  [CUDA memcpy DtoH]
                    0.00%  1.3760us         1  1.3760us  1.3760us  1.3760us  [CUDA memcpy HtoD]
      API calls:   96.34%  2.68781s         1  2.68781s  2.68781s  2.68781s  cudaDeviceSynchronize
                    3.39%  94.549ms         1  94.549ms  94.549ms  94.549ms  cudaMemcpyToSymbol
                    0.24%  6.6836ms         1  6.6836ms  6.6836ms  6.6836ms  cudaMemcpy
                    0.01%  377.72us       114  3.3130us      73ns  176.38us  cuDeviceGetAttribute
                    0.01%  312.57us         1  312.57us  312.57us  312.57us  cudaFree
                    0.01%  183.12us         1  183.12us  183.12us  183.12us  cudaMalloc
                    0.00%  69.573us         1  69.573us  69.573us  69.573us  cudaLaunchKernel
                    0.00%  8.4630us         1  8.4630us  8.4630us  8.4630us  cuDeviceGetName
                    0.00%  6.4770us         1  6.4770us  6.4770us  6.4770us  cuDeviceGetPCIBusId
                    0.00%     732ns         3     244ns      71ns     523ns  cuDeviceGetCount
                    0.00%     447ns         2     223ns      85ns     362ns  cuDeviceGet
                    0.00%     304ns         1     304ns     304ns     304ns  cuDeviceGetUuid
                    0.00%     249ns         1     249ns     249ns     249ns  cuDeviceTotalMem
                    0.00%     232ns         1     232ns     232ns     232ns  cuModuleGetLoadingMode
```

- using Nsight Compute (on TID=04)
  - Block Size vs Execution Time



  - Block Size vs Occupancy



3. Conclusion

- Lesson Learned
  - Understanding Mandelbulb Mathematics
    The Mandelbulb fractal algorithm requires understanding spherical coordinates (theta, phi) and the derivative calculation for distance estimation. Converting the standard complex iteration to 3D spherical coordinates was non-trivial. The orbit trap concept for coloring also required careful implementation to track minimum distances correctly.
  - CUDA Memory and Performance Optimization
    - Using nsight compute for performance tuning
    - Understanding when to use constant memory vs. global memory required experimentation
    - Deciding which functions to inline (__forceinline__) vs. keeping as separate functions involved profiling and performance analysis
    - Optimizing memory coalescing patterns for the raw_image writes required understanding GPU memory architecture
  - Debugging on GPU
    - Limited debugging capabilities made it difficult to verify correctness of mathematical calculations without printf-based debugging, identify memory access issues or kernel crashes or profile where performance bottlenecks actually occur